Student Performance Analytics System

Submitted by: **Krishna Rathod** **COE BATCH 4**

1. Introduction

A college wants to keep the marks of its students in a proper database and analyse them. **Part A** designs a normalized MySQL database (*departments, students, subjects, marks*) with constraints, JOIN queries and stored functions. **Part B** loads the same data from CSV files, cleans and analyses it with NumPy and Pandas, draws four Matplotlib charts and combines everything in a menu-driven Python program. The two parts are independent; the CSV files are the bridge.

Objectives: design a database up to 3NF; apply PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, CHECK and DEFAULT; write JOINs and stored functions; use pd.merge as the Pandas version of JOIN; compute statistics with NumPy/Pandas; present results with Matplotlib; build a validated mini-application.

2. Database design

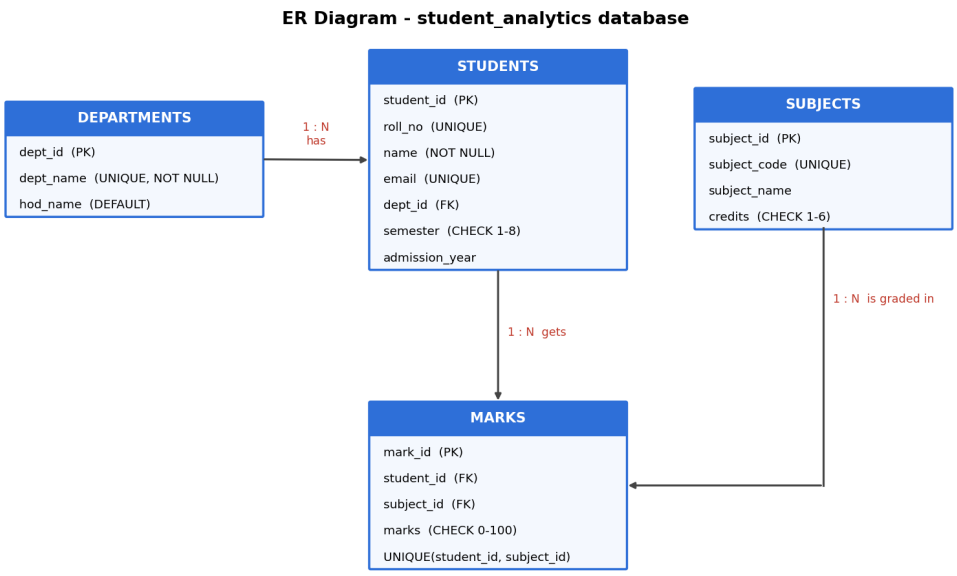


Fig 1: ER diagram. One department has many students; one student has many marks; one subject has many marks (*marks* is the link table between students and subjects).

Table structures and constraints

Table	Columns	Constraints used
departments	dept_id, dept_name, hod_name	PK(dept_id); dept_name NOT NULL + UNIQUE; hod_name DEFAULT 'To be appointed'
subjects	subject_id, subject_code, subject_name, credits	PK; subject_code UNIQUE; credits DEFAULT 3 + CHECK 1-6
students	student_id, roll_no, name, email, dept_id, semester, admission_year	PK; roll_no and email UNIQUE; name NOT NULL; FK(dept_id); semester CHECK 1-8 + DEFAULT 1
marks	mark_id, student_id, subject_id, marks	PK; FK(student_id), FK(subject_id); marks NOT NULL + CHECK 0-100; UNIQUE(student_id, subject_id)

Normalization

1NF: every column holds one atomic value (no lists like 'Maths,Physics') and each table has a primary key. **2NF:** the only composite key is (student_id, subject_id) in *marks*; its only non-key column *marks* depends on the whole key, and student/subject details live in their own tables, so there is no partial dependency. **3NF:** no non-key column depends on another non-key column - department name and HOD are stored only in *departments* (students keeps just dept_id) and subject details only in *subjects*. Every fact is stored once, so there are no update, insert or delete anomalies.

3. SQL work (MySQL 8.0)

Data: 5 departments (Biotechnology has no students, to demonstrate LEFT JOIN), 6 subjects, 30 students and 180 marks (every student in every subject). All queries are in **project.sql**; the script was run end-to-end on MySQL 8.0 and the results below are its actual output.

(a) Marks with student, department and subject name

```
SELECT m.mark_id, s.roll_no, s.name, d.dept_name, sub.subject_name, m.marks
FROM marks m JOIN students s ON s.student_id = m.student_id
             JOIN departments d ON d.dept_id = s.dept_id
             JOIN subjects sub ON sub.subject_id = m.subject_id;
```

(b) Subject-wise average, highest and lowest marks

```
SELECT sub.subject_name, ROUND(AVG(m.marks),2) avg_marks, MAX(m.marks) highest, MIN(m.marks) lowest
FROM marks m JOIN subjects sub ON sub.subject_id = m.subject_id GROUP BY sub.subject_id, sub.subject_name;
```

subject_name	avg_marks	highest	lowest
Data Structures	69.70	98	37
Database Management	67.53	98	38
English Communication	67.20	94	25
Physics	66.23	98	18
Mathematics	65.03	95	34
Python Programming	61.70	90	32

(c) Department-wise topper - CTE + RANK() OVER (PARTITION BY dept_id ORDER BY total DESC)

dept_name	name	total
Civil	Vedant Gaikwad	512
Computer Science	Aarav Sharma	509
Electronics	Vivaan Patil	474
Mechanical	Ananya Iyer	441

(d) Students failing in at least one subject (marks < 40) - WHERE m.marks < 40 with GROUP BY student and GROUP_CONCAT

6 students fail at least one subject: Sneha Kale, Sakshi Rane, Diya Nair, Nidhi Saxena, Mihir Bansal, Shreya Shinde.

(e) Department average using LEFT JOIN

```
SELECT d.dept_name, COUNT(DISTINCT s.student_id) students, ROUND(AVG(m.marks),2) avg_marks
FROM departments d LEFT JOIN students s ON s.dept_id = d.dept_id
                   LEFT JOIN marks m ON m.student_id = s.student_id GROUP BY d.dept_id, d.dept_name;
```

dept_name	avg_marks	students
Computer Science	68.96	8
Electronics	66.31	8
Civil	65.19	7
Mechanical	64.07	7
Biotechnology	NULL	0

Stored functions

get_grade(marks) returns A+ (≥90), A (≥80), B+ (≥70), B (≥60), C (≥50), D (≥40) or F using IF/ELSEIF. **get_percentage(student_id)** returns SUM(marks) / (COUNT(*) x 100) x 100 for one student. **get_result(student_id)** (bonus) returns PASS only if the student has no mark below 40. Usage: `SELECT name, get_percentage(student_id), get_grade(get_percentage(student_id)), get_result(student_id) FROM students;`

Constraint tests (each INSERT is meant to fail) - actual MySQL 8 error messages

Rule tested	INSERT that fails	Error message from MySQL
CHECK marks 0-100	marks = 150	ERROR 3819 (HY000): Check constraint 'chk_marks_range' is violated.
UNIQUE (one mark per subject)	student 1, subject 1 again	ERROR 1062 (23000): Duplicate entry '1-1' for key 'marks.uq_student_subject'
CHECK semester 1-8	semester = 9	ERROR 3819 (HY000): Check constraint 'chk_semester' is violated.
UNIQUE e-mail	existing e-mail	ERROR 1062 (23000): Duplicate entry 'aarav.sharma1@college.edu' for key 'students.email'
FOREIGN KEY	dept_id = 99	ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails (... CONSTRAINT `fk_student_dept` ...)
NOT NULL	name = NULL	ERROR 1048 (23000): Column 'name' cannot be null
DEFAULT	dept without hod_name	Succeeds: hod_name = 'To be appointed'

[Add your own Workbench screenshots of these errors here before submitting.]

4. Python work

CSV files used: departments.csv, students.csv, subjects.csv, marks.csv (exported from MySQL; same data). **Code overview:**

File	What it does
data_handler.py	load_tables() reads the 4 CSVs; merge_tables() joins marks+students+departments+subjects with pd.merge (= SQL query a); load_data() does both; save_marks() writes marks.csv; show_overview() prints head/info/describe.
analysis.py	clean_data() (drop duplicates, to_numeric, fill missing with subject mean); subject_stats_numpy() (np.mean/median/std); make_pivot(); assign_grade() with np.select; build_student_report() (total, %, grade, PASS/FAIL, rank); department_average() with groupby; top_students(); at_risk_students() (bonus).
charts.py	Four Matplotlib charts saved to charts/ as PNG, each with title and axis labels.
main.py	Menu: 1 add/update marks, 2 student report, 3 toppers, 4 analysis, 5 charts, 6 export, 0 exit. Input validation (numbers only, marks 0-100, student/subject must exist) and try/except for FileNotFoundError and PermissionError.

NumPy statistics per subject

subject	mean	median	std_dev
Data Structures	69.7	70.5	16.08
Database Management	67.53	68.5	15.37
English Communication	67.2	70.5	15.72
Physics	66.23	66.5	17.41
Mathematics	65.03	67.0	16.0
Python Programming	61.7	61.0	14.5

Top 5 students (sample of the merged and ranked DataFrame)

rank	roll_no	name	dept_name	total	percentage	grade
1	B244020	Vedant Gaikwad	Civil	512	85.33	A
2	B241001	Aarav Sharma	Computer Science	509	84.83	A
3	B241005	Arjun Joshi	Computer Science	479	79.83	B+
4	B242002	Vivaan Patil	Electronics	474	79.0	B+
5	B242022	Harsh Thakur	Electronics	467	77.83	B+

[Add screenshots of the DataFrame output and the menu program here before submitting.]

5. Results

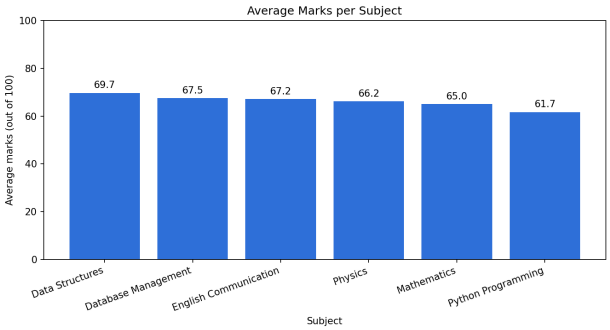


Fig 2: Data Structures has the highest average (69.7) and Python Programming the lowest (61.7); the gap is about 8.0 marks, so no subject is extremely hard.

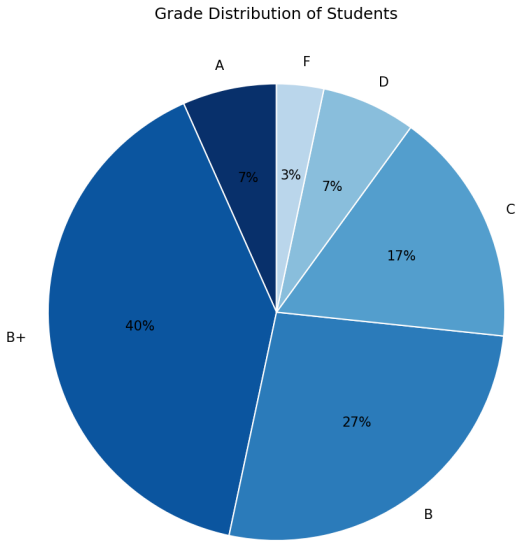


Fig 3: B+ is the most common grade (12 of 30 students); only 2 students got A and 1 got F - most students are in the middle bands.

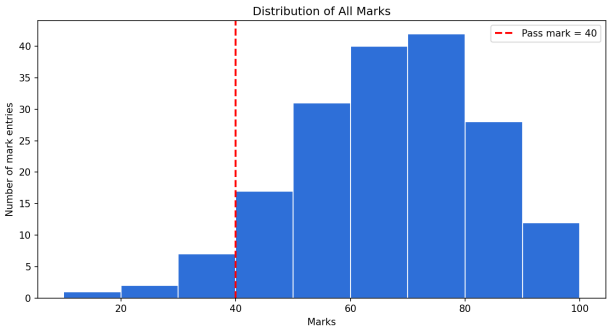


Fig 4: Most marks lie between 50 and 90. Only 10 of 180 entries fall left of the red pass-mark line (40).

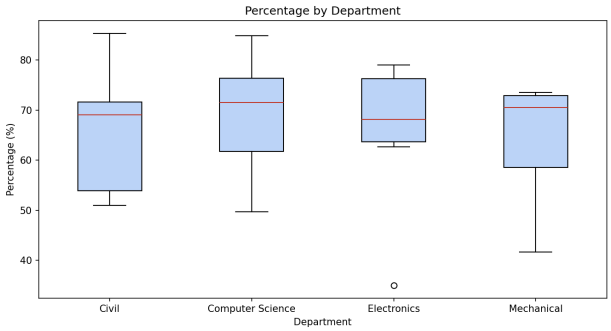


Fig 5: The medians of all four departments are close (about 68-72%). Civil and Computer Science contain the highest scorers (about 85%), and Electronics has one low outlier student.

Other findings: 24 students PASS and 6 FAIL; 7 students are at-risk (below 50% or a failed subject). Overall topper: Vedant Gaikwad (Civil) with 512/600 = 85.33%. **Check:** the Pandas department averages (groupby) equal the MySQL query (e) results: Computer Science 68.96, Electronics 66.31, Civil 65.19, Mechanical 64.07.

6. Conclusion

What I learned: how to normalize a schema and enforce rules with constraints; how JOIN, GROUP BY, CTEs and window functions answer real questions; how stored functions put business rules (grade, result) inside the database; how pd.merge, groupby and pivot_table mirror SQL; how NumPy gives fast statistics; and how to turn numbers into clear Matplotlib charts and a validated menu program.

Problems faced: (1) CHECK constraints are ignored before MySQL 8.0.16, so the version matters. (2) Stored functions need DETERMINISTIC / READS SQL DATA when binary logging is on. (3) A LEFT JOIN returns NULL for a department with no students, which must be matched with NaN in Pandas. (4) Keeping MySQL and CSV data identical - solved by generating both from one script. (5) Input validation: every menu entry needs checks so the program never crashes.

Future improvements: attendance table and correlation with marks (np.corrcoef); semester-wise progress line chart; connect Python to MySQL directly (mysql-connector); a GUI or web dashboard (Streamlit); user login for teachers; PDF report cards per student.

Run instructions: see README.md - run `project.sql` in MySQL, keep the four CSV files next to main.py, then `python main.py`.